capital

Custom Scoping Development

Software Version 2015.1

February 2016

© 2016 Mentor Graphics Corporation
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction **2**
Interfaces **3**
IXComponentScopeSelectionFilter 3
IXHousingScopeSelectionFilter 4
IXComponentSelectionContext 4
IXScopeSelector 5
Exercise **6**
Guidance **11**
Examples **12**

Introduction

This document describes the process for developing Custom Scoping plugins for Capital applications like Capital HarnessXC. This allows you to apply your own scoping rules for the selection of components. If you are **not** interested in developing Custom Scoping, you do not have to read this document. If you are interested in developing Custom Scoping, ensure that you have read and completed the exercises in the PluginDevelopment.pdf document before reading and completing the exercises in this document.

For Capital Harness XC, there are two types of custom scope filters that need implementation. They fall in the following two categories:

- **Components**– these scoping filters allow user to specify rules that should be applied on components during selection process

- **Housing definitions** – these scoping filters allow user to specify rules that should be applied on Housing definitions

Interfaces

IXScopeFilter

All custom scope filters will implement this interface. You should not implement this class directly. You should implement one of its children, for example, IXHousingScopeSelectionFilter, IXComponentScopeSelectionFilter. IXScopingSelector plugin (IXEvaluation) implementation will return a set of filters which contains filters of type IXScopeFilter. It will be used by the application to evaluate scope.

```
@NotToBeImplementedDirectly
public interface IXScopeFilter extends IXFilter
{

}
```

IXComponentScopeSelectionFilter

All custom scope plugins need to implement this interface. Component Scope filter (IXScopeFilter) implementation will return a set of library objects that are applicable in the context provided.

```
public interface IXComponentScopeSelectionFilter<o extends IXLibraryObject> extends IXScopeFilter
{

    Collection<o> apply(Collection<o> objs, IXComponentSelectionContext sContext);
}
```

IXHousingScopeSelectionFilter

All custom Scope Plugins need to implement this interface. Housing Scope filter (IXScopeFilter) implementation will return a set of housing definitions that are applicable in the context provided.

```
public interface IXHousingScopeSelectionFilter<o extends IXLibraryHousingDefinition> extends IXScopeFilter
{

    Collection<o> apply(Collection<o> objs, IXComponentSelectionContext sContext);
}
```

IXComponentSelectionContext

This object allows any IXScopeFilter to have access to useful objects to evaluate the scope.

```
public interface IXComponentSelectionContext
{

    /**
     * Returns the design to which the library object belongs type i.e. IXLogicDesign
     *
     * @return IXDesign
     */
    IXDesign getDesign();

    /**
     * Returns the Project to which the library object belongs
     *
     * @return IXProject
     */
    IXProject getProject();
}
```

IXScopeSelector

Any Scope Plugin should implement this interface and Capital Applications will call the plugin to get scoping filters (IXScopeFilter) which needs to be applied to evaluate the scope of component. Two types of filters are currently supported, IXHousingScopeSelectionFilter and IXComponentScopeSelectionFilter

```
@OnlyOneImplementationAllowed
public interface IXScopeSelector<o extends IXScopeFilter> extends IXEvaluation<o>
{


}
```

```
public @interface OnlyOneImplementationAllowed
{


}
```

Exercise

In this exercise, you will develop a custom scoping plugin for the Library Driven Scoping Java plugin. i.e., the scope codes defined on Library part should satisfy the scope codes defined on design.

You need to be familiar with Capital HarnessXC and CapitalLibrary to do this exercise.

1) The next step is to create a new custom scoping plugin.

    a. Ensure that your Java development environment has been set-up. If you have completed the Java plugin exercise then it should already be setup. If you have not, then you should stop this exercise and complete it now.

    b. Open a UNIX/DOS console. Ensure that the environment variables required for Java, ANT and Capital have been setup in this console. Cd into the following folder within the Capital installation:

       doc/plugin/examples/Java/src/com/example/plugin/scoping.

    c. In the above folder, there example scoping plugins. Here is the location of the file as seen from the Capital installation folder:

       The folder src is the directory where all the example Java plugin source code has been placed. This should exist already.

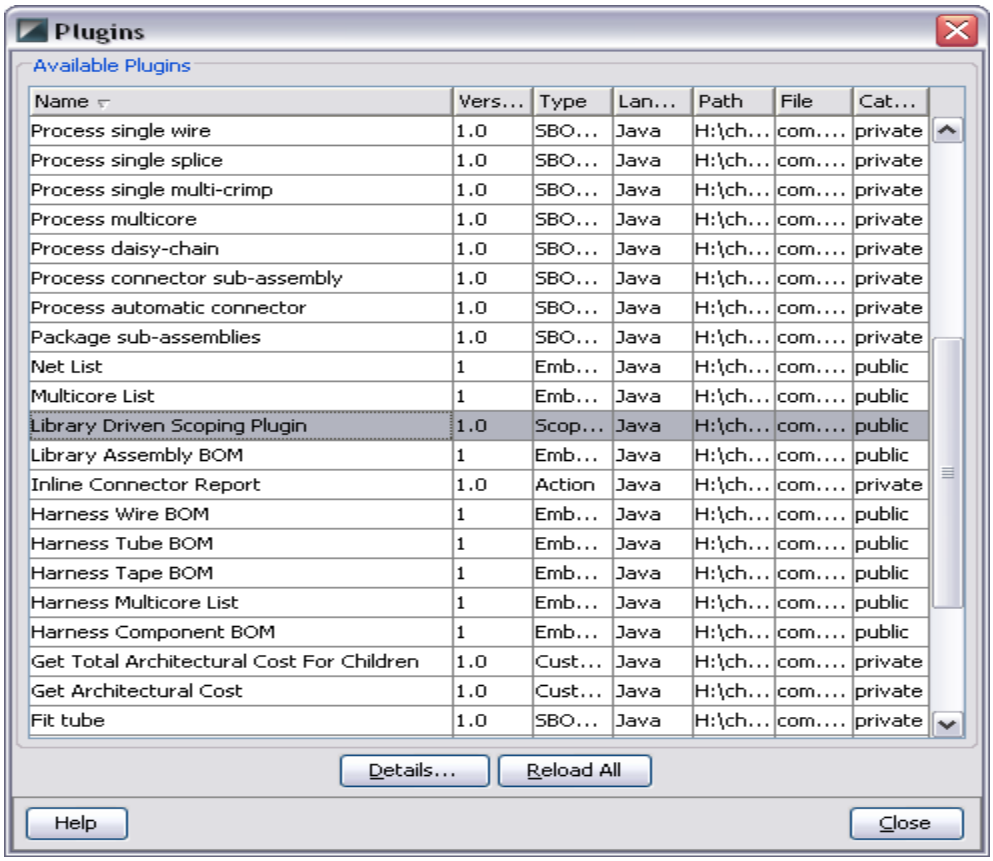
    d. Change directory into “doc/plugin/examples/Java”
    e. Run the command ant LibraryDrivenScoping
    f. You will see example.jar build in plugins/Examples if it was not present.
    g. If examples.jar is already existing it will be update with the librarydrivenscoping plugin.

    h. If everything is fine, you will eventually get a “BUILD SUCCESSFUL” message. If you do not, you will need to check the environment variables or check for (and fix) errors in the LibraryDrivenScoping files and try again.

    i. Go back to CapitalLibrary

    j. From the main menu, click on the File -> Plugins… menu

k. Press the [Reload All] button on the Plugins dialog. You will see your new plugin appear in the list with the other available plugins. See the selected row in the image below:



Name	Vers...	Type	Lan...	Path	File	Cat...
Process single wire	1.0	SBO...	Java	H:\ch...	com....	private
Process single splice	1.0	SBO...	Java	H:\ch...	com....	private
Process single multi-crimp	1.0	SBO...	Java	H:\ch...	com....	private
Process multicore	1.0	SBO...	Java	H:\ch...	com....	private
Process daisy-chain	1.0	SBO...	Java	H:\ch...	com....	private
Process connector sub-assembly	1.0	SBO...	Java	H:\ch...	com....	private
Process automatic connector	1.0	SBO...	Java	H:\ch...	com....	private
Package sub-assemblies	1.0	SBO...	Java	H:\ch...	com....	private
Net List	1	Emb...	Java	H:\ch...	com....	public
Multicore List	1	Emb...	Java	H:\ch...	com....	public
Library Driven Scoping Plugin	1.0	Scop...	Java	H:\ch...	com....	public
Library Assembly BOM	1	Emb...	Java	H:\ch...	com....	public
Inline Connector Report	1.0	Action	Java	H:\ch...	com....	private
Harness Wire BOM	1	Emb...	Java	H:\ch...	com....	public
Harness Tube BOM	1	Emb...	Java	H:\ch...	com....	public
Harness Tape BOM	1	Emb...	Java	H:\ch...	com....	public
Harness Multicore List	1	Emb...	Java	H:\ch...	com....	public
Harness Component BOM	1	Emb...	Java	H:\ch...	com....	public
Get Total Architectural Cost For Children	1.0	Cust...	Java	H:\ch...	com....	private
Get Architectural Cost	1.0	Cust...	Java	H:\ch...	com....	private
Fit tube	1.0	SBO...	Java	H:\ch...	com....	private

Leave this dialog open for now.

2) Similarly you can write your own plugin, you have to change the build.xml file accordingly.

3) First, you should start by creating some test data.

a. Start Capital Manager

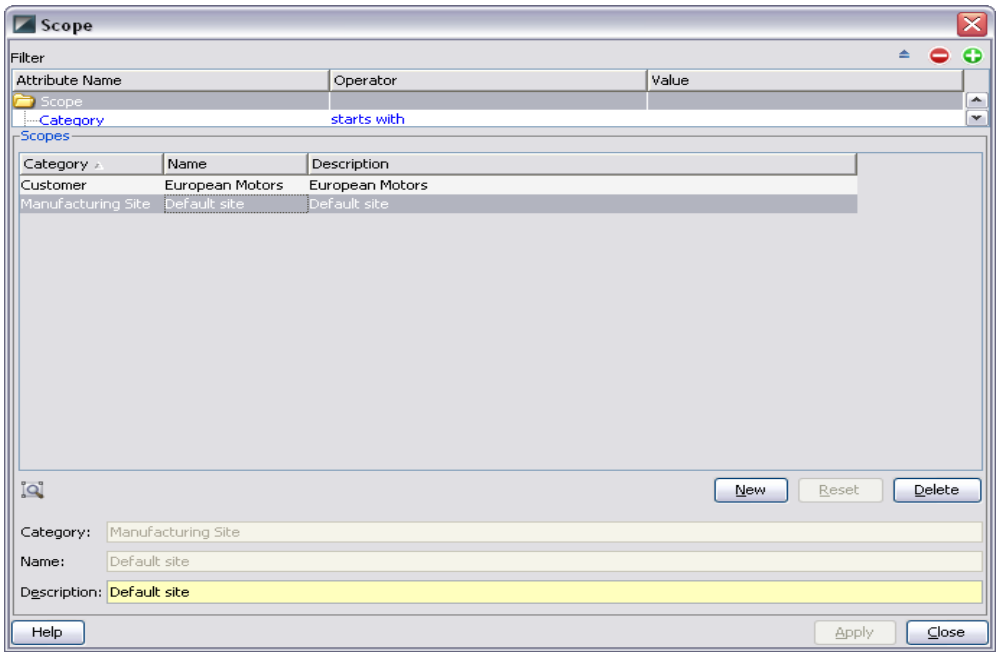
b. Start Capital Project

c. Import any Qucikstart data. e.g., data\xml\project\ Quick Start – Automotive Generative.xml.

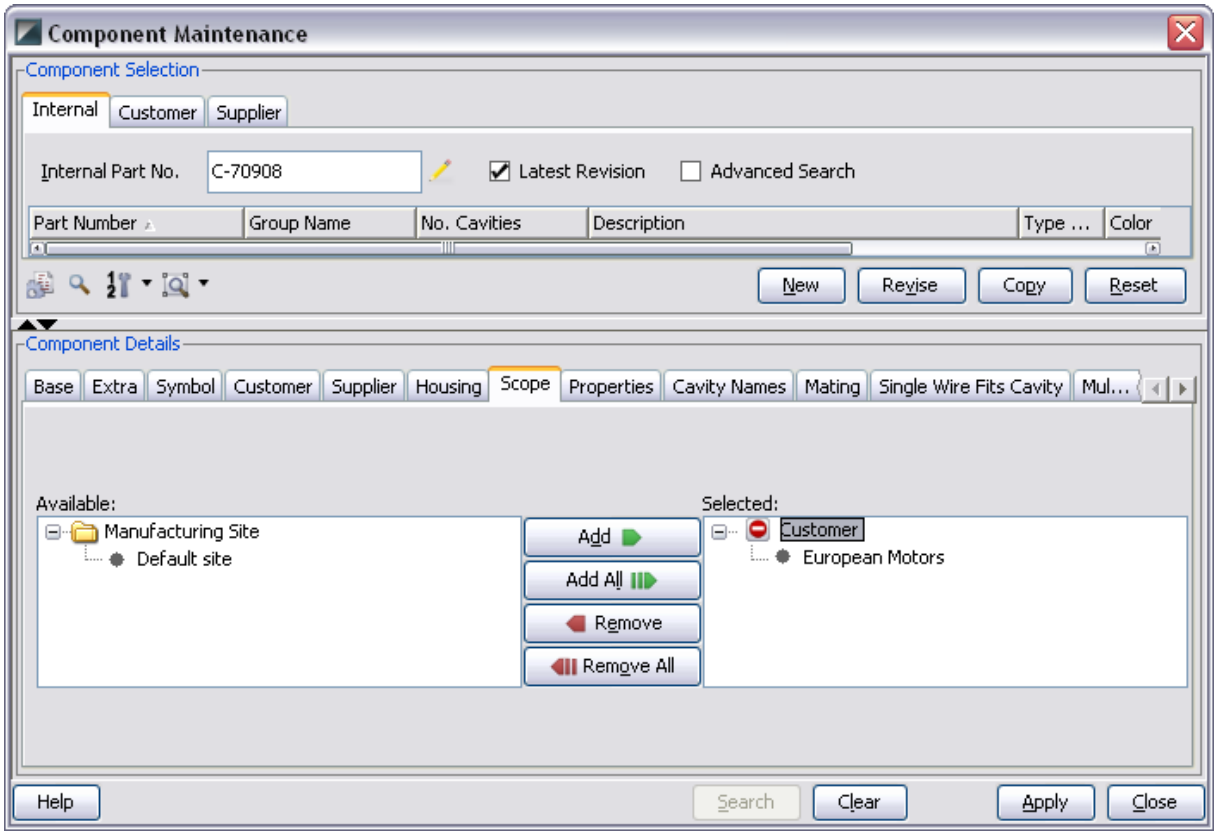
d. Start Capital Library

e. Import Quick start Library from data\xml\library\ Quick Start - Library.xml

f. In Capital Library go to codes->scopes and define the following scope codes.

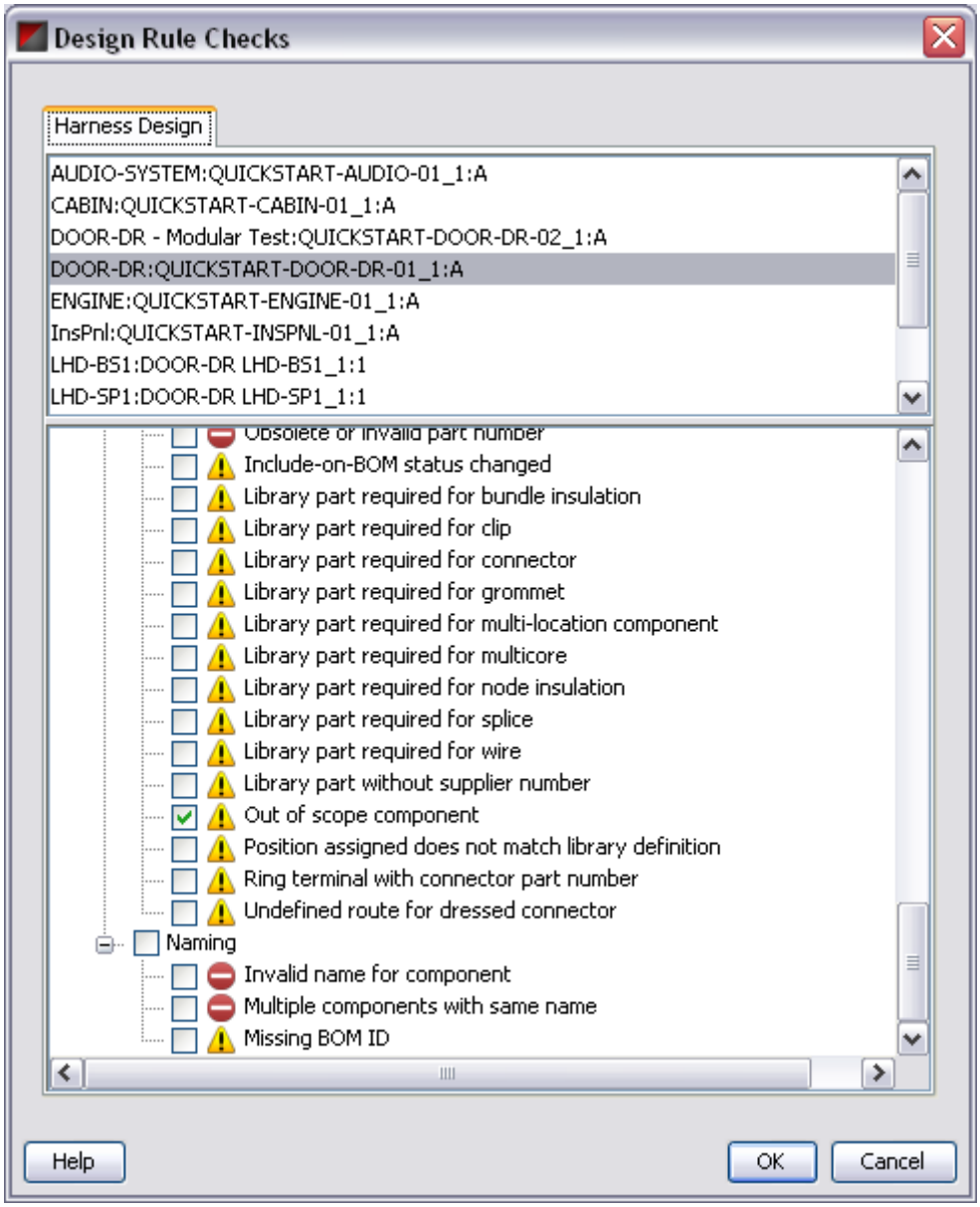


g. go to components->component maintenance
h. define some scope values on component “C-70908” as shown below



i. open “CapitalHarnessXC”
j. edit Design “QUICKSTART-DOOR-DR-01_1”
k. edit properties -> scopes

l. Save changes to design
m. Run the DRC as shown below



n. Now look at the checks log, you should see component “C-70908” failing with message component out of scope.

Guidance

Always develop custom scoping in Java for performance. Also ensure that any debug messages are removed or disabled before your custom scoping Plugin is deployed.

If you have developed your own custom scoping plugins, and they are in use at your company, care should be taken when maintaining those plugins. You should avoid renaming the class or moving the class to another package. If you do this, any previously created design will no longer be able to find the custom scoping plugins if you deploy an updated plugin that has a new class name or package.

Any given time there should only one scoping related plugin existing otherwise Capital will throw and error and fall back to default scoping behavior. All the scoping plugins should have scoping filters defined for both housings and components otherwise scoping behavior may be in consistent.

Examples

At this point, you should have a good idea how to implement this type of plugin. Some examples have been provided to help you. These examples can be found in the following location within the Capital installation:

```
doc/plugin/examples/Java/src/com/example/plugin/scoping
```

The following examples are good starting points for Capital HarnessXC:

- **LibraryDrivenScoping–** This is an example code provided to find out whether the scope code defined library part satisfies with design.

    1. LibraryDrivenScopeSelector.java: custom scope plugin should implement IXScopeSelector.

    2. LibraryDrivenComponentScopeFilter.java: custom scope plugin should define this filter to compute the scoping for objects of type IXLibraryObject.

    3. LibraryDrivenHousingScopeFilter.java: custom scope plugins should define this filter to compute the scoping for objects of type IXLibraryHousingDefinition.

    4. LibraryDrivenScopeValidator.java: this file contains the business logic for computing the applicability of components and housing definitions.

- **LibraryAndDesignDrivenScoping –** As the name suggests this is an example code provided to find out whether the scope codes defined library part satisfies with design and scope codes defined on design satisfies with library part.

    1. LibraryAndDesgingDrivenScopeSelector.java: custom scope plugin should implement IXScopeSelector.

    2. LibraryAndDesginDrivenComponentScopeFilter.java: custom scope plugin should define this filter to compute the scoping for objects of type IXLibraryObject

    3. LibraryAndDesginDrivenHousingScopeFilter.java: custom scope plugins should define this filter to compute the scoping for objects of type IXLibraryHousingDefinition.

    4. LibraryAndDesignDrivenScopeValidator.java: this file contains the business logic for computing the applicability of components and housing definitions.

You can copy and modify the examples to implement your own plugins if you wish. However, you need to check the license and copyright information at the start of the files before you do this.

The examples provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples).